P2 পোস্টার প্রেজেন্টেশন: বক্তৃতা ও গাইড

Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection

Mashud Hasan (22299053)
Nirnoy Biswas (24241185)
Md. Sabbir Hossain (22299038)
Al Imran (231011035)
Imtiaz Zaman Sami (23101551)

সুপারভাইজার: Md. Sabbir Ahmed, Senior Lecturer
কো-সুপারভাইজার: Naima Tahsin Nodi, Adjunct Lecturer

Department of Computer Science and Engineering, Brac University

পর্ব ক. এই ফাইলটা কীভাবে ব্যবহার করবে

পর্ব খ-এর বক্তৃতা স্থির গতিতে প্রায় ৬ মিনিটের। পর্ব গ পোস্টারের প্রতিটা ব্লক আর প্রতিটা ফিগার সহজ ভাষায় ব্যাখ্যা করে, যাতে টিমের যে কেউ যেকোনো অংশ নিয়ে প্রশ্নের উত্তর দিতে পারে। পর্ব ঘ থেকে চ হলো চিট-শিট: সংখ্যা, শব্দের সহজ মানে, আর সম্ভাব্য প্রশ্ন।

সময়	অংশ	পোস্টারের কোন ব্লক দেখাবে	প্রস্তাবিত বক্তা
০:০০ - ০:৪৫	শুরু আর সমস্যা	টাইটেল, Abstract, Problem Statement	বক্তা ১
০:৪৫ - ১:৩৫	উদ্দেশ্য আর literature	Research Objectives, Literature Review	বক্তা ১
১:৩৫ - ২:২৫	ডেটাসেট আর preprocessing	Data set টেবিল, preprocessing ডায়াগ্রাম	বক্তা ২
২:২৫ - ৩:৪০	পদ্ধতি, মডেল, framework	Methodology-র তিনটা ডায়াগ্রাম	বক্তা ৩
৩:৪০ - ৫:১৫	ফলাফল	ধূসর Result Analysis বক্স	বক্তা ৪
৫:১৫ - ৬:০০	উপসংহার আর ভবিষ্যৎ কাজ	Conclusion & Future Work	বক্তা ৫

বক্তা ভাগ করা শুধু একটা প্রস্তাব; একজনও পুরো বক্তৃতা দিতে পারে।

টিপস: পোস্টারের সামনে নয়, পাশে দাঁড়াও। কোনো ফিগার নিয়ে বলার আগে সেটার দিকে দেখাও। গ্রাফের এক লাইনের মূল কথাটা আগে বলো, তারপর একটা সংখ্যা। কোনো সংখ্যা ভুলে গেলে শুধু মূল কথাটাই যথেষ্ট। প্রশ্ন কোনো সীমাবদ্ধতা নিয়ে হলে সেটা স্বীকার করো, আর বলো ভবিষ্যৎ কাজে সেটা কীভাবে সমাধান হবে।

পর্ব খ. বক্তৃতা

শান্ত, স্থির গতিতে পড়ো (মিনিটে প্রায় ১৩০-১৪০ শব্দ)। কমলা লাইনগুলো মঞ্চের নির্দেশনা: জোরে পড়বে না। বোল্ড শব্দগুলোতে জোর দাও।

০:০০ - ০:৪৫ | শুরু আর সমস্যা

[টাইটেলের পাশে দাঁড়াও। হাসিমুখে দর্শকদের দিকে তাকাও, পোস্টারের দিকে নয়।]

শুভ সকাল। আমরা মাশুদ, নির্নয়, সাব্বির, ইমরান আর সামি। আমাদের থিসিস হলো আর্থিক লেনদেনে জালিয়াতি ধরার জন্য একটা **drift-aware continuous learning MLOps framework**। আমাদের সুপারভাইজার মো. সাব্বির আহমেদ স্যার, আর কো-সুপারভাইজার নাঈমা তাহসিন নদী ম্যাম।

ব্যাংকগুলো জালিয়াতি লেনদেন ধরতে machine learning মডেল ব্যবহার করে। সাধারণত মডেলকে একবার ট্রেন করা হয়, একবার পরীক্ষা করা হয়। কিন্তু বাস্তবে দুটো সমস্যা হয়। প্রথমত, প্রতারকরা নিয়মিত কৌশল বদলায়, তাই ডেটা ধীরে ধীরে বদলে যায় আর ভালো মডেলও দুর্বল হয়ে পড়ে। একে বলে **concept drift**। দ্বিতীয়ত, কোনো লেনদেন যে জালিয়াতি ছিল, ব্যাংক সেটা জানতে পারে কয়েক সপ্তাহ পরে, যখন গ্রাহক অভিযোগ করেন। একে বলে **label delay**। তাই মডেল সময়মতো নিজের ভুল দেখতে পায় না।

[গোলাপি Problem Statement বক্সের দিকে দেখাও।]

তাই আমাদের প্রশ্নটা সহজ: **একটা fraud মডেলকে কখন আবার ট্রেন করা উচিত, আর খারাপ নতুন মডেলকে চালু হওয়া থেকে কীভাবে আটকানো যায়?**

০:৪৫ - ১:৩৫ | উদ্দেশ্য আর literature review

[Research Objectives, তারপর Literature Review দেখাও।]

আমাদের পাঁচটা উদ্দেশ্য ছিল: চালু মডেল কীভাবে খারাপ হয় আর drift alarm সেটা ধরে কি না তা মাপা; label delay কীভাবে retraining-এর লাভ কমায় তা মাপা; নির্দিষ্ট সময়সূচিতে retraining আর drift-triggered retraining-কে ন্যায্যভাবে তুলনা করা; একটা পূর্ণ MLOps framework বানিয়ে ইচ্ছাকৃত ত্রুটি দিয়ে পরীক্ষা করা; আর সবকিছু তিনটা ডেটাসেটে আবার করা।

Literature থেকে: Dal Pozzolo ও সহকর্মীরা দেখিয়েছেন fraud-এর লেবেল দেরিতে আসে। Menezes ও Filho দেখিয়েছেন স্বাভাবিক drift-এ fraud মডেলের F1-score ৪০ শতাংশ পর্যন্ত কমে যায়, কিন্তু তাঁরা retraining পরীক্ষা করেননি। Shakil ও সহকর্মীরা MLOps-এ drift-triggered retraining বানিয়েছেন, কিন্তু শুধু কৃত্রিম drift আর সঙ্গে সঙ্গে আসা লেবেল দিয়ে। তাই **গবেষণার ফাঁক** হলো: আসল drift আর দেরিতে আসা লেবেলসহ fraud-এর retraining পদ্ধতিগুলো কেউ ন্যায্যভাবে পরীক্ষা করেনি।

১:৩৫ - ২:২৫ | ডেটাসেট আর preprocessing

[ডেটাসেট টেবিল, তারপর preprocessing ডায়াগ্রাম দেখাও।]

আমরা Kaggle থেকে তিনটা পাবলিক ডেটাসেট ব্যবহার করেছি। **IEEE-CIS** হলো আসল অনলাইন পেমেন্ট: ছয় মাসে প্রায় ৫ লাখ ৯০ হাজার লেনদেন, ৩.৫ শতাংশ fraud। **Sparkov** হলো simulated কার্ড পেমেন্ট: দুই বছরে ১৮ লাখ লেনদেন, মাত্র আধা শতাংশ fraud। **BAF** হলো ব্যাংক অ্যাকাউন্ট খোলার আবেদন: আট মাসে ১০ লাখ সারি, প্রায় ১ শতাংশ fraud। তিনটা আলাদা ডেটাসেট দেখায় যে আমাদের ফলাফল একটা ডেটাসেটের ভাগ্য নয়।

Preprocessing-এ আমরা ফাইলগুলো জোড়া লাগিয়ে পরিষ্কার করেছি, ব্যক্তিগত তথ্য সরিয়েছি, আর ঘণ্টা, বয়স, দূরত্ব, কার্ডের ইতিহাসের মতো ফিচার বানিয়েছি, শুধু **আগের** লেনদেন থেকে, যাতে মডেল কখনো ভবিষ্যৎ দেখতে না পায়। তারপর সবকিছু সময় অনুযায়ী সাজিয়েছি, কখনো shuffle করিনি। প্রথম অর্ধেক দিয়ে প্রথম মডেল ট্রেন হয়, আর বাকিটা সপ্তাহে সপ্তাহে replay করা হয়, ঠিক বাস্তবের মতো।

২:২৫ - ৩:৪০ | পদ্ধতি, মডেল আর framework

[Workflow ডায়াগ্রাম (৮টা বক্স) দেখাও।]

আমাদের গবেষণার আটটা ধাপ ছিল: literature review, ডেটা প্রস্তুত, একটা static baseline, ০ থেকে ৬০ দিনের label delay-সহ retraining simulation, confidence interval আর পাঁচটা random seed দিয়ে পরিসংখ্যানিক যাচাই, তারপর framework বানানো, ত্রুটি দিয়ে পরীক্ষা, আর সব ডেটাসেটে পুনরাবৃত্তি।

[Model architecture ডায়াগ্রাম দেখাও।]

আমাদের মডেল হলো **LightGBM**। এটা একের পর এক অনেকগুলো ছোট decision tree বানায়, আর প্রতিটা নতুন tree আগেরগুলোর ভুল ঠিক করে। সবগুলোর আউটপুট যোগ করে ০ থেকে ১-এর মধ্যে একটা fraud সম্ভাবনা বানানো হয়। আমরা এটা বেছেছি কারণ এটা দ্রুত, টেবিল ডেটায় শক্তিশালী, আর ফাঁকা মান নিজেই সামলায়। আমরা নিজেরাই ট্রেন করেছি; কোনো pre-trained মডেল নেই।

[Framework ডায়াগ্রাম দেখাও, হাত দিয়ে তীরগুলো অনুসরণ করো।]

আমাদের framework সাপ্তাহিক একটা লুপে চলে। লেনদেনগুলো একটা API দিয়ে score করা হয়। লেবেল দেরিতে একটা label store-এ আসে। **Monitor** যে লেবেলগুলো এসেছে তাতে পারফরম্যান্স মাপে। **Retrain policy** প্রতি ১৪ দিনে retrain করে, আর পারফরম্যান্স হঠাৎ ভেঙে পড়লে একটা safety net আছে। Trainer দুটো training window চেষ্টা করে, শেষ ৬০ দিন আর পুরো ইতিহাস, আর ভালোটা রাখে। তারপর **promotion gate** নতুন মডেলকে চালু মডেলের সাথে না-দেখা ডেটায় তুলনা করে, আর খারাপ না হলে তবেই চালু করে। MLflow প্রতিটা ভার্সন রাখে, তাই আগের মডেলে ফেরা যায়, আর একটা dashboard প্রতিটা সিদ্ধান্ত দেখায়।

৩:৪০ - ৫:১৫ | ফলাফল

[প্রথম result ফিগার দেখাও (ওপরে লাইন, নিচে বার)।]

আমাদের মূল মাপকাঠি **PR-AUC**: মডেল আসল fraud-গুলোকে তালিকার কতটা ওপরে রাখতে পারে। প্রথম ফলাফল: drift alarm পারফরম্যান্সের সাথে মেলে না। IEEE-CIS-এ পারফরম্যান্স ০.৫৮ থেকে ০.৩৯-এ নেমে যায়, কিন্তু drift score কখনো alarm লাইন পার হয় না। এটা একটা **missed alarm**। BAF-এ drift score বিশাল, কিন্তু পারফরম্যান্স সমান থাকে। এটা একটা **false alarm**।

[Label-delay ফিগার দেখাও।]

দ্বিতীয়: label delay retraining-এর লাভ খেয়ে ফেলে। IEEE-CIS-এ লেবেল সঙ্গে সঙ্গে এলে retraining PR-AUC ০.০৭৮ বাড়ায়, কিন্তু ৬০ দিন দেরিতে এলে মাত্র ০.০১০। Window-ও গুরুত্বপূর্ণ: Sparkov-এ ছোট window উল্টো ক্ষতি করে।

[Window-selection বার দেখাও।]

তৃতীয়: আমাদের স্বয়ংক্রিয় window বাছাই প্রতিটা ডেটাসেটে সেরা নির্দিষ্ট window-এর সমান হয়েছে, আগে থেকে না জেনেই কোনটা ঠিক।

[Gate ফিগার দেখাও।]

চতুর্থ: আমরা ইচ্ছা করে ডেটা নষ্ট করেছি, যেমন লেবেল এলোমেলো করে। Gate ছাড়া পারফরম্যান্স ০.০৮০ পর্যন্ত কমে গেছে। Gate থাকলে মাত্র ০.০১২, কারণ খারাপ মডেলগুলো আটকে গেছে।

আর সুপারভাইজারের পরামর্শমতো, আমরা দুটো পদ্ধতিকে **সমান সংখ্যক retrain** দিয়ে তুলনা করেছি। লেবেল দেরিতে এলে schedule তখনো drift trigger-এর সমান বা ভালো ছিল, তাই এটা শুধু বেশিবার retrain করার কারণে জেতে না।

৫:১৫ - ৬:০০ | উপসংহার আর ভবিষ্যৎ কাজ

[Conclusion & Future Work দেখাও।]

শেষে বলি: লেবেল দেরিতে এলে drift alarm আসল ক্ষতি ধরতে ব্যর্থ হতে পারে, আর ঠিক training window-সহ একটা সহজ schedule বেশি নির্ভরযোগ্য। আমাদের কাজে **drift-aware** মানে দেরিতে আসা লেবেলে পারফরম্যান্স দেখে drift বোঝা, লেবেল-ছাড়া alarm দিয়ে নয়। Window selection আর promotion gate retraining-কে খাপ খাওয়ানো আর নিরাপদ করে, আর ফলাফল তিনটা ডেটাসেটেই টিকেছে।

সুপারভাইজার আমাদের scoring API-এর load test করতেও বলেছিলেন। প্রথম পরীক্ষায় একটা ধীরগতির জায়গা ধরা পড়ে; সেটা ঠিক করার পর API সেকেন্ডে প্রায় **১,৩০০টা রিকোয়েস্ট** সামলায়, আর একটা লেনদেন প্রায় এক মিলিসেকেন্ডে score করে, score হুবহু একই রেখে।

এরপর আমরা Sparkov-এ equal-budget পরীক্ষা শেষ করব। পরে দ্রুত investigator feedback, অন্য মডেল আর লাইভ ব্যবহার পরীক্ষা করতে চাই। ধন্যবাদ। আপনাদের প্রশ্ন শুনতে আমরা প্রস্তুত।

পর্ব গ. পোস্টারের প্রতিটা অংশ সহজ ভাষায়

পোস্টারের প্রতিটা ব্লক সহজ কথায়। প্রতিটা ফিগারের জন্য: কী দেখায়, কীভাবে পড়বে, মনে রাখার সংখ্যা, আর এক লাইনে কী বলবে।

১. Abstract আর Problem Statement

কাজটা কী: একটা সিস্টেম যা চালু হওয়ার পরও fraud মডেলকে সুস্থ রাখে। এটা মডেলকে নজরে রাখে, নির্দিষ্ট সময়সূচিতে retrain করে, কতটা পুরোনো ডেটা থেকে শিখবে তা বাছাই করে, আর একটা পরীক্ষা (promotion gate) পার হলে তবেই নতুন মডেল চালু করে।

কেন দরকার: বেশিরভাগ গবেষণা (১) random split-এ পরীক্ষা করে, যা অতীত আর ভবিষ্যৎ মিশিয়ে drift লুকিয়ে ফেলে, (২) ধরে নেয় লেবেল সঙ্গে সঙ্গে আসে, যা fraud-এ কখনো হয় না, আর (৩) কৃত্রিমভাবে ঢোকানো drift-এ পরীক্ষিত drift detector দিয়ে retraining শুরু করে। আমরা এর বদলে বাস্তবসম্মত অবস্থায় পরীক্ষা করেছি।

২. Research Objectives

#	উদ্দেশ্য	পোস্টারে উত্তর কোথায়
১	চালু মডেল কীভাবে খারাপ হয়, আর drift মাপ সেটা ধরে কি না	ফলাফল ফিগার ১ (decay)
২	Label delay কীভাবে retraining-এর লাভ সীমিত করে	ফলাফল ফিগার ২ (delay)
৩	Schedule আর drift-triggered retraining-এর ন্যায্য তুলনা	Conclusion (equal-budget পয়েন্ট)
৪	Framework বানানো আর ত্রুটি ঢুকিয়ে পরীক্ষা	Framework ডায়াগ্রাম, ফলাফল ফিগার ৩ ও ৪
৫	তিনটা ডেটাসেটে পুনরাবৃত্তি	প্রতিটা ফলাফল ফিগারে তিনটা প্যানেল

৩. Literature Review (পোস্টারের তিনটা পেপার)

পেপার	তাঁরা কী পেয়েছেন	কী বাকি ছিল (আমাদের ফাঁক)
Dal Pozzolo et al. (২০১৮)	Fraud-এর লেবেল দেরিতে আসে (verification latency)	MLOps retraining পদ্ধতি পরীক্ষা করেননি
Menezes ও Filho (২০২৫)	IEEE-CIS-এ স্বাভাবিক drift-এ fraud মডেলের F1 ৪০% পর্যন্ত কমে	ক্ষতি মেপেছেন, retraining পরীক্ষা করেননি
Shakil et al. (২০২৫)	MLOps pipeline-এর ভেতরে drift-triggered retraining	শুধু কৃত্রিম drift আর সঙ্গে সঙ্গে আসা লেবেল

এক লাইনে ফাঁক: আসল drift আর label delay-সহ fraud-এর retraining পদ্ধতির কোনো ন্যায্য পরীক্ষা ছিল না।

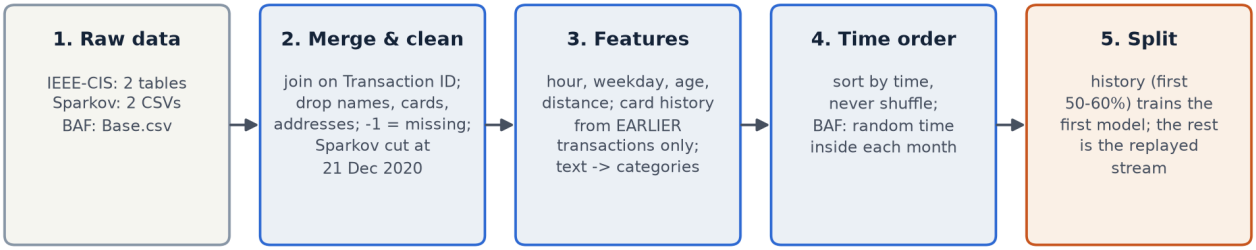
৪. Data set টেবিল

	IEEE-CIS	Sparkov	BAF
কী এটা	আসল ই-কমার্স পেমেন্ট (Vesta, Kaggle প্রতিযোগিতা)	Simulated ক্রেডিট-কার্ড পেমেন্ট (Sparkov generator)	ব্যাংক অ্যাকাউন্ট খোলার আবেদন (NeurIPS ২০২২)
সারি	৫,৯০,৫৪০	১৮,০১,৯৬৯	১০,০০,০০০
সময়	১৮২ দিন	৭২০ দিন	৮ মাস
Fraud-এর হার	৩.৫০%	০.৫৩%	১.১০%
ফিচার	৪৩১	১৬	২৯

	IEEE-CIS	Sparkov	BAF
কেন বেছেছি	আসল drift; মডেল সত্যিই খারাপ হয়	লম্বা সময় আর আসল টাইমস্ট্যাম্প (সুপারভাইজার দ্বিতীয় টাইমস্ট্যাম্পযুক্ত ডেটাসেট চেয়েছিলেন)	মাসে মাসে drift রাখার জন্য বানানো; ভিন্ন ধরনের fraud

কীভাবে পড়বে: প্রতিটা কলাম একটা ডেটাসেট। Fraud-এর হার দেখায় fraud কত বিরল: Sparkov-এ প্রতি ১,০০০ লেনদেনে মাত্র প্রায় ৫টা। 'ফিচার' হলো মডেল কয়টা ইনপুট কলাম ব্যবহার করে। তিনটাই টেবিল আকারের ডেটা, যেখানে একটা হ্যাঁ/না fraud লেবেল আছে।

৫. Preprocessing ডায়াগ্রাম

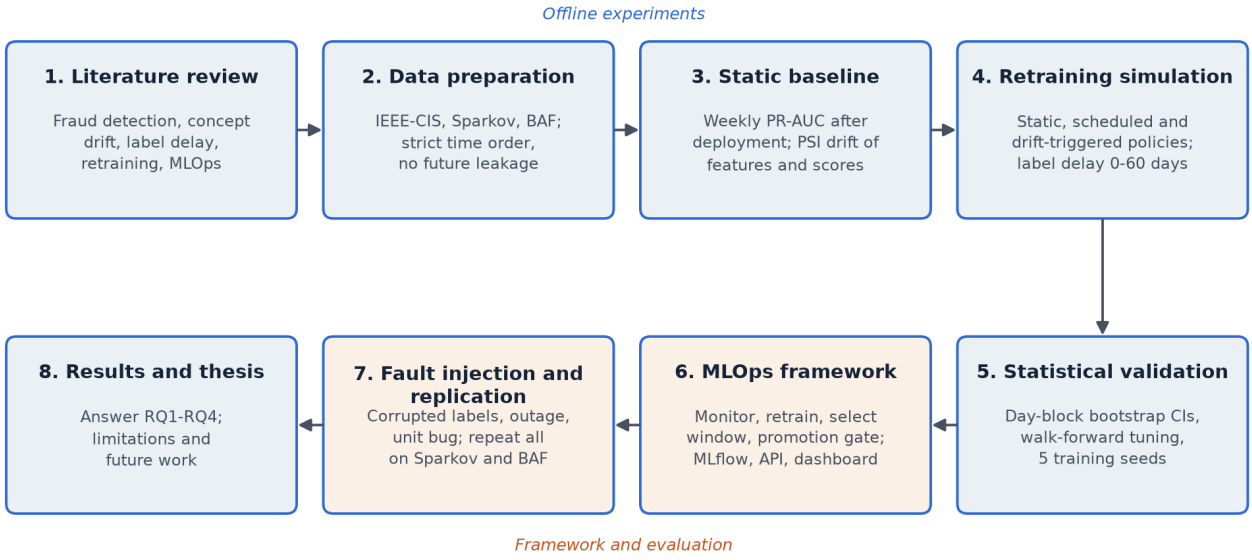


output: one table per dataset with TransactionID, TransactionDT (seconds), isFraud and the model features

কী দেখায়	ডাউনলোড করা কাঁচা ফাইল থেকে পরিষ্কার টেবিল পর্যন্ত পাঁচটা ধাপ, যা প্রতিটা পরীক্ষায় ব্যবহার হয়।
কীভাবে পড়বে	বাঁ থেকে ডানে। (১) কাঁচা ফাইল। (২) জোড়া ও পরিষ্কার: IEEE-এর দুটো টেবিল জোড়া, নাম আর কার্ড নম্বর সরানো, BAF-এ −১-কে ফাঁকা ধরা, Sparkov ২১ ডিসেম্বর ২০২০-এ কাটা কারণ সেখানে fraud থেমে যায়। (৩) ফিচার বানানো (ঘণ্টা, সপ্তাহের দিন, বয়স, দূরত্ব, কার্ডের ইতিহাস), শুধু আগের লেনদেন থেকে। (৪) সময় অনুযায়ী সাজানো, কখনো shuffle নয়। (৫) ভাগ: প্রথম ৫০-৬০% দিয়ে প্রথম মডেল, বাকিটা সপ্তাহে সপ্তাহে replay।
কেন গুরুত্বপূর্ণ	Shuffle করলে মডেল ভবিষ্যৎ দেখে ফেলত আর drift লুকিয়ে যেত। শুধু আগের লেনদেন থেকে ফিচার বানালে data leakage হয় না।

> **এক লাইনে বলো:** আমরা কঠোর সময়ক্রম রেখেছি, যাতে পরীক্ষাটা একটা আসল ব্যাংকের মতো আচরণ করে।

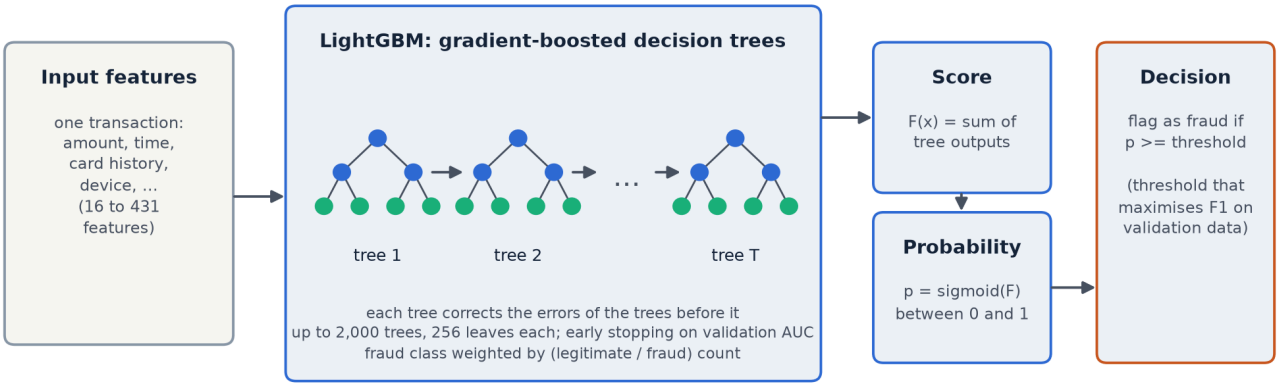
৬. গবেষণার Workflow



কী দেখায়	পুরো প্রজেক্টের আটটা ধাপ।
কীভাবে পড়বে	ওপরের সারি বাঁ থেকে ডানে (offline পরীক্ষা): literature, ডেটা, static baseline, retraining simulation। তারপর নিচে নেমে ডান থেকে বাঁয়ে (framework ও মূল্যায়ন): পরিসংখ্যানিক যাচাই, MLOps framework, ত্রুটি ঢোকানো ও পুনরাবৃত্তি, ফলাফল।
মূল শব্দ	**Static baseline**: একবার ট্রেন, কখনো আপডেট নয়, আর দেখা কীভাবে খারাপ হয়। **Bootstrap CI**: দিনগুলো বারবার নতুন করে বেছে দেখা একটা পার্থক্য কতটা নিশ্চিত। **Walk-forward tuning**: শুধু অতীত দিয়ে সেটিং বাছাই, তারপর পরের সময়ে পরীক্ষা। **৫টা seed**: ৫টা আলাদা random শুরু দিয়ে ট্রেনিং, যাতে ফলাফল ভাগ্য না হয়।

> **এক লাইনে বলো:** আগে সমস্যাটা মেপেছি, তারপর framework বানিয়েছি, তারপর সেটা ভাঙার চেষ্টা করেছি।

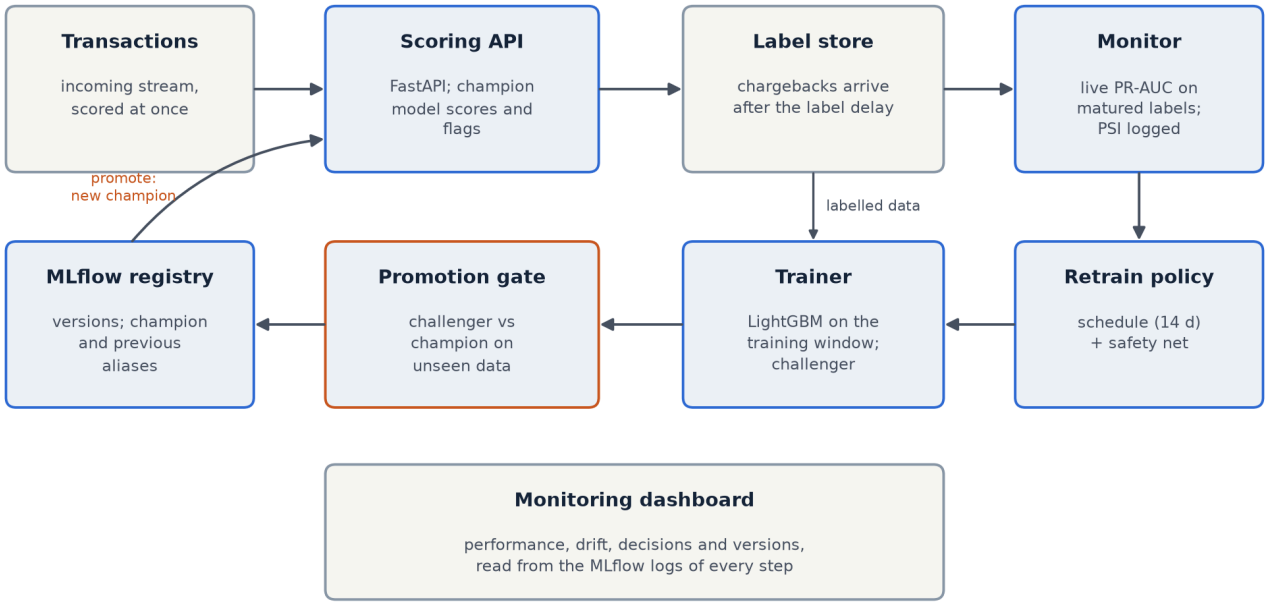
৭. Model architecture (LightGBM)



কী দেখায়	একটা লেনদেন কীভাবে fraud সিদ্ধান্তে পরিণত হয়।
কীভাবে পড়বে	ইনপুট ফিচার (১৬ থেকে ৪৩১টা কলাম) অনেকগুলো ছোট decision tree-তে যায়। Tree ২ শেখে tree ১-এর ভুল থেকে, tree ৩ শেখে tree ২ থেকে, এভাবে (একে বলে 'gradient boosting')। আউটপুটগুলো যোগ হয়ে একটা score হয়, score থেকে ০ থেকে ১-এর মধ্যে সম্ভাবনা হয়, আর সম্ভাবনা একটা সীমার (threshold) ওপরে গেলে লেনদেনটা fraud হিসেবে চিহ্নিত হয়।
সেটিং	সর্বোচ্চ ২,০০০টা tree, প্রতিটায় ২৫৬টা পাতা, learning rate ০.০৫, validation-এ উন্নতি থামলে আগেই থামে। বিরল fraud শ্রেণিকে বেশি গুরুত্ব (weight) দেওয়া হয়, কৃত্রিম fraud সারি বানানো হয় না (SMOTE নয়)।
কেন LightGBM	শত শত বার retrain করার মতো দ্রুত, টেবিল ডেটায় সবচেয়ে পরিচিত ভালো মডেল, ফাঁকা মান আর category নিজেই সামলায়। শূন্য থেকে ট্রেন করা, pre-trained নয়।

> **এক লাইনে বলো:** অনেকগুলো ছোট tree, প্রতিটা আগেরটার ভুল ঠিক করে, মিলে একটা fraud সম্ভাবনা দেয়।

৮. প্রস্তাবিত Framework



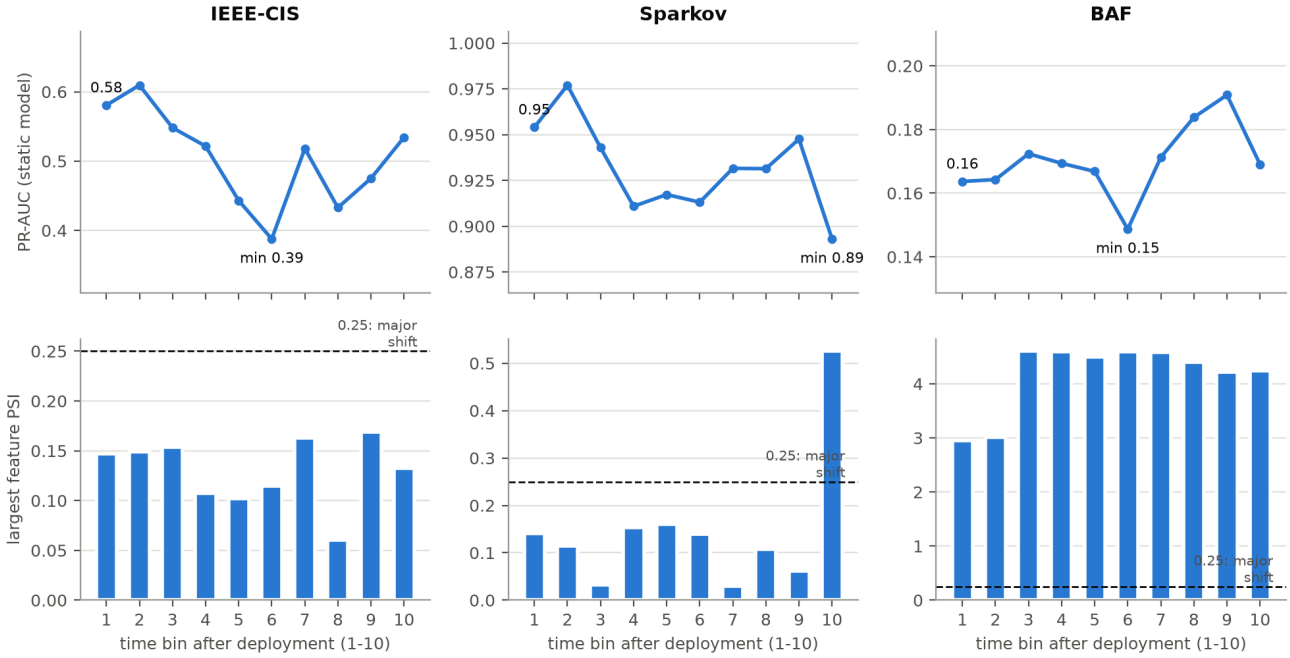
কী দেখায়	আমাদের MLOps সিস্টেম, আর প্রতি সপ্তাহে ডেটা কীভাবে এর মধ্য দিয়ে যায়।
কীভাবে পড়বে	তীর অনুসরণ করো। **Transactions** score হয় **Scoring API** (FastAPI) দিয়ে। আসল লেবেল দেরিতে **Label store**-এ আসে। **Monitor** যে লেবেল এসেছে তাতে লাইভ PR-AUC মাপে। **Retrain policy** প্রতি ১৪ দিনে retrain করে, সঙ্গে safety net। **Trainer** নতুন মডেল (challenger) বানায়। **Promotion gate** একে চালু মডেলের (champion) সাথে তুলনা করে। পাস করলে **MLflow** এটা রেজিস্টার করে আর এটা নতুন champion হয়। **Dashboard** সব দেখায়।
Safety net	লাইভ PR-AUC স্বাভাবিকের চেয়ে ৩০% নিচে নামলে, বা random অনুমানের কাছাকাছি নামলে আগেভাগেই retrain করে।
Window selection	প্রতিটা retrain-এ দুটো challenger ট্রেন হয়, শেষ ৬০ দিন আর পুরো ইতিহাস; যেটা সবচেয়ে নতুন না-দেখা লেবেলে ভালো করে, সেটা রাখা হয়।
Promotion gate	নতুন মডেল চালু মডেলের চেয়ে বড়জোর ০.০১ PR-AUC খারাপ হলে তবেই চালু হয়; আসল প্রোডাকশন ফিচার দিয়ে নতুন করে মাপা হয়।

> **এক লাইনে বলো:** একটা সাপ্তাহিক লুপ: নজর রাখা, সিদ্ধান্ত, retrain, যাচাই, তারপর চালু।

পর্ব গ (চলমান). ফলাফলের ফিগারগুলো

> **গ্রাফের আগে: PR-AUC কী?** মডেল প্রতিটা লেনদেনকে একটা fraud score দেয়। বেশি থেকে কম অনুযায়ী সাজাও। PR-AUC (০ থেকে ১) বলে আসল fraud-গুলো ওই তালিকার কতটা ওপরে আছে। ১ মানে নিখুঁত। Random অনুমান করা মডেল পায় প্রায় fraud-এর হারের সমান (IEEE-CIS-এ ০.০৩৫, BAF-এ ০.০১১)। তাই BAF-এর ০.১৬ আসলে অনুমানের চেয়ে প্রায় ১৫ গুণ ভালো। আমরা accuracy-র বদলে PR-AUC ব্যবহার করি, কারণ ৯৯% লেনদেন স্বাভাবিক হলে 'কখনো fraud নয়' বলা মডেল ৯৯% accurate হয়, কিন্তু কোনো কাজের নয়।
>
> **PSI কী?** Population Stability Index: training সময়ের তুলনায় একটা ফিচারের মান কতটা বদলেছে। ০.১-এর নিচে = স্থির, ০.১ থেকে ০.২৫ = মাঝারি, ০.২৫-এর ওপরে = বড় পরিবর্তন (সাধারণ alarm লাইন)।
>
> **Error bar** (ওপরে-নিচে টুপিওয়ালা সরু দাগ): ৯৫% confidence interval। দাগটা শূন্য না ছুঁলে পার্থক্যটা নির্ভরযোগ্য।

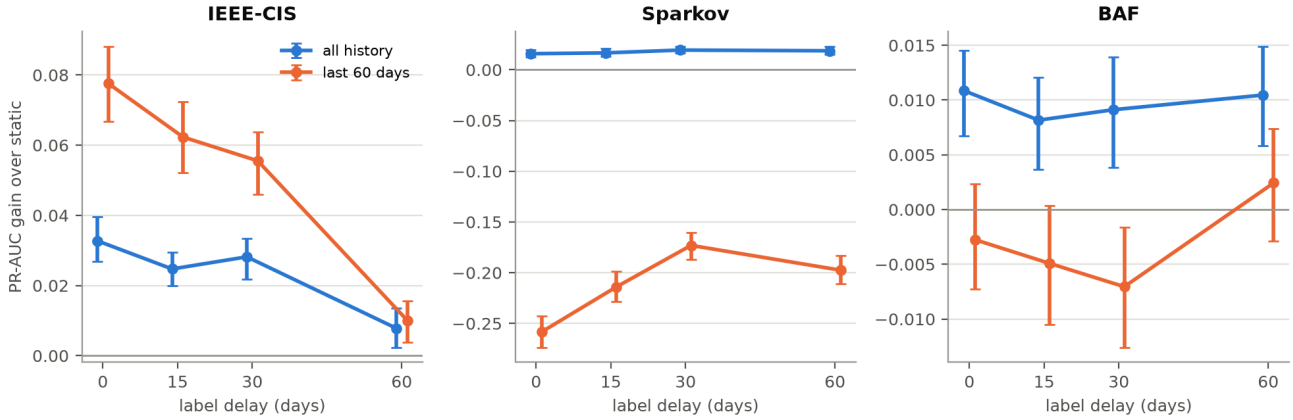
ফলাফল ১. Drift মাপ পারফরম্যান্সের সাথে মেলে না



প্রশ্ন	মডেল খারাপ হলে drift alarm কি সেটা ধরে?
কী দেখায়	প্রতিটা ডেটাসেটের জন্য একটা কলাম। **ওপরের সারি**: একবার ট্রেন করা, কখনো আপডেট না করা মডেলের PR-AUC, চালুর পরের ১০টা সময়ভাগে। **নিচের সারি**: সবচেয়ে বেশি বদলানো ফিচারের PSI। ০.২৫-এর ড্যাশড লাইন হলো 'বড় পরিবর্তন' alarm।
কীভাবে পড়বে	একই কলামের ওপর আর নিচ তুলনা করো। Alarm কাজ করলে লাইন যখন নামছে, ঠিক তখনই বারগুলো ড্যাশড লাইনের ওপরে উঠত।
সংখ্যা	**IEEE-CIS**: PR-AUC ০.৫৮ থেকে ০.৩৯-এ নামে, কিন্তু PSI ০.১৭-এর নিচেই থাকে, কখনো ০.২৫ ছোঁয় না: **missed alarm**। **BAF**: PSI ৩ থেকে ৪.৫, alarm লাইনের প্রায় ১৫ গুণ, কিন্তু PR-AUC ০.১৫-০.১৯-এর আশেপাশে স্থির: **false alarm**। **Sparkov**: PR-AUC উঁচুতেই থাকে (০.৮৯-০.৯৮); PSI শুধু শেষ সময়ভাগে লাইন পার হয়।
কেন এই গ্রাফ	সময়ের সাথে পরিবর্তন দেখাতে লাইন; প্রতিটা সময়ে পরিমাণ দেখাতে বার। দুই সারি, কারণ PR-AUC আর PSI-এর মাপের পরিসর আলাদা।

> **এক লাইনে বলো:** IEEE-CIS-এ drift alarm আসল ক্ষতি ধরতে পারেনি, আর BAF-এ ভুয়া alarm দিয়েছে।

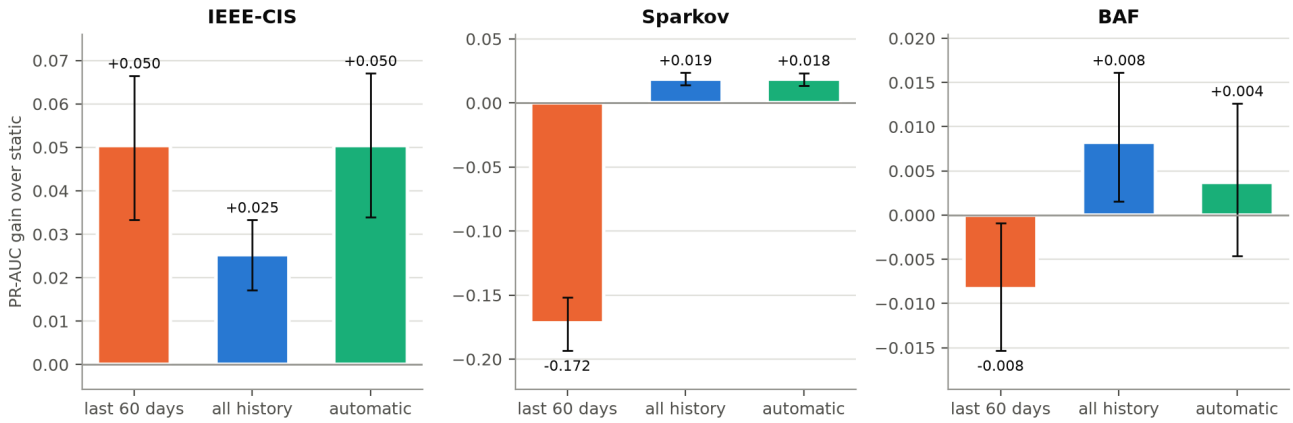
ফলাফল ২. Label delay retraining-এর লাভ কমিয়ে দেয়



প্রশ্ন	লেবেল দেরিতে এলে retraining তখনো কতটা সাহায্য করে?
কী দেখায়	X অক্ষ: label delay (০, ১৫, ৩০, ৬০ দিন)। Y অক্ষ: কখনো-আপডেট-না-হওয়া মডেলের চেয়ে retraining কতটা ভালো (PR-AUC লাভ)। নীল = পুরো ইতিহাসে ট্রেন, কমলা = শেষ ৬০ দিনে ট্রেন। Error bar = ৯৫% confidence।
কীভাবে পড়বে	শূন্য লাইনের ওপরে মানে retraining সাহায্য করে; নিচে মানে ক্ষতি করে। নামতে থাকা লাইন মানে লেবেল যত দেরিতে আসে, লাভ তত কমে।
সংখ্যা	**IEEE-CIS**, শেষ ৬০ দিন: ০ দিনে +০.০৭৮, ৬০ দিনে নেমে +০.০১০। পুরো ইতিহাস: +০.০৩৩ থেকে +০.০০৮। **Sparkov**: শেষ ৬০ দিন শূন্যের অনেক নিচে (−০.১৭ থেকে −০.২৬), কারণ ৬০ দিনে খুব কম fraud থাকে; পুরো ইতিহাস প্রায় +০.০২। **BAF**: পুরো ইতিহাস প্রায় +০.০১; শেষ ৬০ দিন শূন্যের কাছাকাছি বা সামান্য নিচে।
কেন এই গ্রাফ	Delay বাড়ার সাথে প্রবণতা লাইনে সবচেয়ে পরিষ্কার; error bar দেখায় পার্থক্যগুলো আসল কি না।

> **এক লাইনে বলো:** লেবেল যত দেরিতে আসে, retraining তত কম সাহায্য করে, আর ঠিক window ডেটাসেটের ওপর নির্ভর করে।

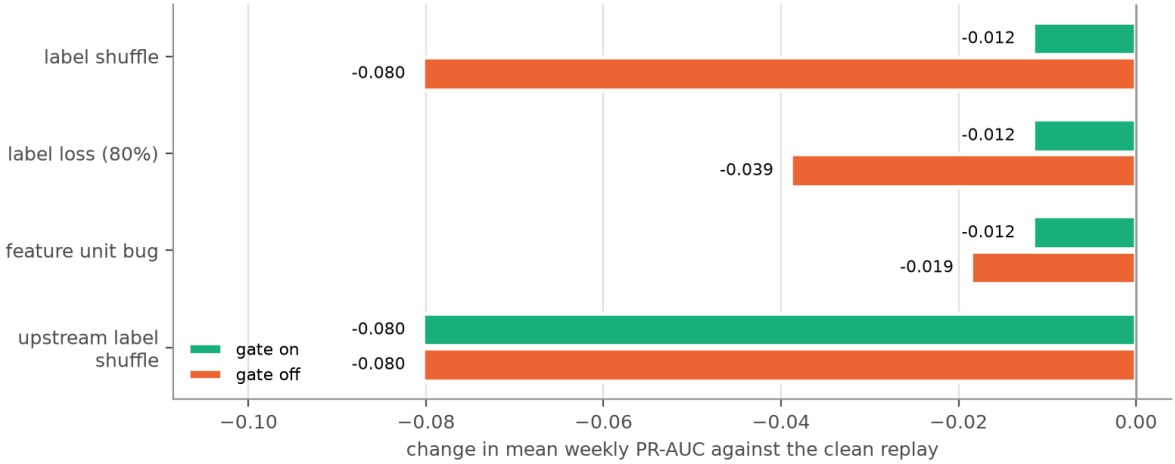
ফলাফল ৩. স্বয়ংক্রিয় window বাছাই



প্রশ্ন	Framework কি নিজেই training window বাছতে পারে?
কী দেখায়	৩০ দিনের label delay-সহ পুরো framework-এর replay। প্রতিটা ডেটাসেটে তিনটা বার: সবসময় শেষ ৬০ দিন (কমলা), সবসময় পুরো ইতিহাস (নীল), স্বয়ংক্রিয় বাছাই (সবুজ)। উচ্চতা = কখনো-আপডেট-না-হওয়া মডেলের চেয়ে PR-AUC লাভ।
কীভাবে পড়বে	সেরা নির্দিষ্ট window প্রতিটা ডেটাসেটে আলাদা। দেখো সবুজ বার কমলা আর নীলের মধ্যে যেটা লম্বা, তার কাছাকাছি কি না।
সংখ্যা	**IEEE-CIS**: ৬০ দিন +০.০৫০, পুরো ইতিহাস +০.০২৫, স্বয়ংক্রিয় +০.০৫০। **Sparkov**: ৬০ দিন −০.১৭২, পুরো ইতিহাস +০.০১৯, স্বয়ংক্রিয় +০.০১৮। **BAF**: ৬০ দিন −০.০০৮, পুরো ইতিহাস +০.০০৮, স্বয়ংক্রিয় +০.০০৪ (এর error bar শূন্য পার হয়, তাই মোটামুটি কোনো পরিবর্তন নেই)।
কেন এই গ্রাফ	কয়েকটা বিকল্প পাশাপাশি তুলনায় বার সবচেয়ে পরিষ্কার।

> **এক লাইনে বলো:** স্বয়ংক্রিয় বাছাই আগে থেকে না জেনেই সেরা নির্দিষ্ট window-এর সমান হয়েছে, আর Sparkov-এর বড় ক্ষতি এড়িয়েছে।

ফলাফল 8. Promotion gate



প্রশ্ন	Gate কি খারাপ মডেলকে চালু হওয়া থেকে আটকায়?
কী দেখায়	আমরা ইচ্ছা করে চারভাবে training ডেটা নষ্ট করেছি। প্রতিটার জন্য, পরিষ্কার রানের তুলনায় গড় সাপ্তাহিক PR-AUC কতটা কমেছে। সবুজ = gate চালু, কমলা = gate বন্ধ। লম্বা বার = বেশি ক্ষতি।
ত্রুটিগুলো	**Label shuffle**: fraud লেবেল এলোমেলো করা। **Label loss (৮০%)**: বেশিরভাগ লেবেল হারানো। **Feature unit bug**: একটা ফিচার ভুল এককে (যেমন ডলারের বদলে সেন্ট)। **Upstream label shuffle**: উৎসেই লেবেল নষ্ট, তাই দুই মডেলই ভুল লেবেলে বিচার হয়।
সংখ্যা	Gate বন্ধ: −০.০৮০, −০.০৩৯, −০.০১৯। Gate চালু: প্রতিবার মাত্র −০.০১২। Upstream shuffle: দুই ক্ষেত্রেই −০.০৮০।
শেষ সারি কেন সমান	Gate হাতে থাকা লেবেল দিয়েই মডেল যাচাই করে। লেবেল উৎসেই ভুল হলে gate-ও ধোঁকা খায়। সেই ক্ষেত্রে safety net আর ডেটার মান যাচাই দরকার। এটা সৎভাবে বলো; এতে বোঝা যায় তুমি সীমাটা বোঝো।

> **এক লাইনে বলো:** Gate নষ্ট ডেটায় ট্রেন হওয়া প্রতিটা খারাপ মডেল আটকেছে, ক্ষতি ০.০৮০ থেকে কমিয়ে ০.০১২ করেছে।

৯. Conclusion আর Future Work

পোস্টারের পয়েন্ট	সহজ ভাষায় মানে
লেবেল দেরিতে এলে drift indicator আসল অবনতি ধরতে ব্যর্থ হতে পারে	সাধারণ alarm ইনপুট দেখে, ফলাফল নয়। IEEE-CIS-এ আসল পতন ধরতে পারেনি, আর BAF-এ মিথ্যা বিপদসংকেত দিয়েছে।
Drift-aware = দেরিতে আসা লেবেলে পারফরম্যান্স দেখে drift ট্র্যাক করা	আমরা drift নিয়ে এখনো ভাবি, কিন্তু বিচার করি যে লেবেল এসেছে তাতে মডেল আসলে কেমন করছে তা দিয়ে।
সমান retraining বাজেট	সুপারভাইজারের প্রশ্ন: schedule কি শুধু বেশি retrain করে বলেই জেতে? আমরা দুটোকে সমান সংখ্যক retrain দিয়েছি। IEEE-CIS-এ (৬০ দিনের window) ২৭টা জোড়ার ১৮টায় schedule ভালো ছিল আর trigger কখনো নির্ভরযোগ্যভাবে ভালো ছিল না; BAF-এ ৩০ দিনের delay-তেও schedule এগিয়ে ছিল।
Window selection আর promotion gate	Retraining প্রতিটা ডেটাসেটের সাথে খাপ খায় আর নষ্ট ডেটা থেকে নিরাপদ থাকে।
API load test (শেষ)	সুপারভাইজারের অনুরোধ। প্রথম পরীক্ষায় দেখা গেল ইনপুট বদলাতেই প্রায় সব সময় যাচ্ছে (সেকেন্ডে মাত্র প্রায় ১৪টা রিকোয়েস্ট)। ঠিক করার পর, score হুবহু একই রেখে: সেকেন্ডে প্রায় ১,৩০০টা রিকোয়েস্ট, একটা লেনদেনে প্রায় ১ ms, একসাথে ১০০ জন client-এ p95 latency ৮২ ms, কোনো error নেই।
এরপর	Sparkov-এ equal-budget পরীক্ষা; কয়েকটা worker process-সহ সার্ভারে load test আবার চালানো।
পরে	দ্রুত investigator feedback, অন্য মডেল, ব্যাখ্যার (SHAP) পর্যবেক্ষণ, লাইভ ব্যবহার।

পর্ব ঘ. মনে রাখার সংখ্যা

কী	সংখ্যা
ডেটাসেট	৩টা: IEEE-CIS (৫,৯০,৫৪০ সারি), Sparkov (১৮,০১,৯৬৯), BAF (১০,০০,০০০)
Fraud-এর হার	৩.৫%, ০.৫৩%, ১.১%
পরীক্ষা করা label delay	০, ১৫, ৩০, ৬০ দিন
Retrain সময়সূচি	প্রতি ১৪ দিনে; safety net সর্বোচ্চ প্রতি ৭ দিনে একবার
Training window	শেষ ৬০ দিন, বা পুরো ইতিহাস; স্বয়ংক্রিয়ভাবে বাছাই
Safety net	স্বাভাবিকের চেয়ে ৩০% পতন, বা random অনুমানের কাছাকাছি
Promotion gate	নতুন মডেল বড়জোর ০.০১ PR-AUC খারাপ হতে পারে
IEEE-CIS-এ অবনতি	PR-AUC ০.৫৮ থেকে ০.৩৯, PSI কখনো ০.২৫-এর ওপরে নয়
Retraining লাভ, IEEE-CIS	+০.০৭৮ (delay নেই) থেকে +০.০১০ (৬০ দিন delay)
Gate-এর প্রভাব	gate ছাড়া ক্ষতি −০.০৮০, gate-সহ −০.০১২
API load test	সেকেন্ডে প্রায় ১,৩০০ রিকোয়েস্ট; লেনদেনপ্রতি ১ ms; ১০০ client-এ p95 ৮২ ms (ঠিক করার আগে: সেকেন্ডে ১৪টা)
নির্ভরযোগ্যতা	৫টা random seed, walk-forward tuning, bootstrap confidence interval
টুল	Python, LightGBM, pandas, scikit-learn, MLflow, FastAPI, Kaggle

পর্ব ঙ. সহজ ভাষায় শব্দগুলো

শব্দ	সহজ মানে
Concept drift	সময়ের সাথে ডেটা বা জালিয়াতির ধরন বদলায়, তাই পুরোনো মডেল খারাপ হয়।
Label delay	কোনো লেনদেন fraud ছিল কি না, সেটা আমরা জানি কয়েক সপ্তাহ পরে।
MLOps	Machine learning মডেল বাস্তবে চালানো: চালু করা, নজর রাখা, আপডেট করা, আগের ভার্সনে ফেরা।
Retraining	নতুন ডেটায় নতুন একটা মডেল ট্রেন করা।
Scheduled retraining	যা-ই ঘটুক, নির্দিষ্ট সময়ে (প্রতি ১৪ দিনে) retrain।
Drift-triggered retraining	শুধু তখনই retrain, যখন কোনো alarm বলে ডেটা বা পারফরম্যান্স বদলেছে।
Training window	নতুন মডেল কতটা পুরোনো ডেটা থেকে শেখে।
Champion / challenger	চালু মডেল / নতুন ট্রেন হওয়া মডেল যে তার জায়গা নিতে চায়।
Promotion gate	চালু হওয়ার আগে challenger খারাপ নয় কি না, তার পরীক্ষা।
MLflow	প্রতিটা মডেল ভার্সন আর তার মাপ সংরক্ষণের টুল।
FastAPI	লেনদেন score করার ওয়েব সার্ভিস বানানোর টুল।
Replay	পুরোনো ডেটা সপ্তাহে সপ্তাহে চালানো, যেন এখনই ঘটছে।
Fault injection	নিরাপত্তা পরীক্ষার জন্য ইচ্ছা করে ডেটা নষ্ট করা।

পর্ব চ. সম্ভাব্য প্রশ্ন (সংক্ষিপ্ত উত্তর)

প্রশ্ন	উত্তর
Accuracy কেন নয়?	Fraud প্রায় ১%, তাই 'কখনো fraud নয়' বললেই ৯৯% accurate, কিন্তু কোনো কাজের নয়। PR-AUC সতর্কতার তালিকার ওপরে fraud ধরার ওপর নজর দেয়।
BAF-এর PR-AUC (০.১৬) এত কম কেন?	BAF-এর fraud আলাদা করা খুব কঠিন আর মাত্র ১.১%। Random অনুমান পায় ০.০১১, তাই ০.১৬ প্রায় ১৫ গুণ ভালো।
Pre-trained মডেল ব্যবহার করেছ?	না। LightGBM প্রতিটা ডেটাসেটে শূন্য থেকে, শত শত বার ট্রেন করা হয়েছে।
LightGBM কেন, deep learning নয় কেন?	টেবিল ডেটায় boosted tree সাধারণত সবচেয়ে শক্তিশালী, আর বহুবার retrain করার মতো দ্রুত। অন্য মডেল ভবিষ্যৎ কাজ।
ডেটা shuffle করোনি কেন?	Shuffle করলে ভবিষ্যৎ ট্রেনিংয়ে মিশে যায় আর drift লুকিয়ে যায়। আসল ব্যাংক সবসময় অতীত থেকে শিখে ভবিষ্যৎ অনুমান করে।
Schedule কেন drift trigger-কে হারায়?	Trigger-এর পারফরম্যান্স পড়ার প্রমাণ দরকার, আর সেই প্রমাণ আসে কয়েক সপ্তাহ পুরোনো লেবেল থেকে। Schedule অপেক্ষা না করেই মডেলকে নতুন রাখে।
Schedule কি শুধু বেশি retrain করে বলেই ভালো?	না। সমান সংখ্যক retrain-এও, লেবেল দেরিতে এলে schedule trigger-এর সমান বা ভালো ছিল।
IEEE-CIS-এ ৬০ দিন, কিন্তু Sparkov-এ পুরো ইতিহাস কেন?	IEEE-CIS দ্রুত বদলায়, তাই পুরোনো ডেটা ভুল শেখায়। Sparkov স্থির আর fraud বিরল, তাই ৬০ দিনে যথেষ্ট fraud থাকে না।
Gate ধোঁকা খেলে কী হয়?	লেবেল উৎসেই নষ্ট হলে দুই মডেলকেই একই রকম দেখায়। তখন safety net আর ডেটা যাচাই সেটা সামলায়।
তোমাদের কাজে নতুন কী?	Label delay-সহ বাস্তবসম্মত পরীক্ষা, ন্যায্য equal-budget তুলনা, আর এমন একটা framework যাতে আছে দেরিতে আসা লেবেলে monitoring, স্বয়ংক্রিয় window বাছাই, আর প্রোডাকশন ফিচারে নতুন করে score করা gate।
এটা কি real time?	আমরা পুরোনো ডেটা সপ্তাহে সপ্তাহে replay করি, কিন্তু scoring API। আসল: load test-এ এটা একটা লেনদেনের উত্তর প্রায় ১ ms-এ দিয়েছে আর সেকেন্ডে প্রায় ১,৩০০টা রিকোয়েস্ট সামলেছে। লাইভ stream পরের ধাপ।
Load test-এ কী পেলে?	প্রথমে সেকেন্ডে মাত্র ১৪টা রিকোয়েস্ট, কারণ JSON-কে pandas টেবিলে বদলাতে ৫৭ ms লাগছিল। এখন ইনপুট সরাসরি সংখ্যায় বদলাই: score একই, প্রায় ১০০ গুণ দ্রুত।
p95 latency কী?	১০০টা রিকোয়েস্টের ৯৫টা যে সময়ের মধ্যে শেষ হয়। গড়ের চেয়ে এটা বেশি গুরুত্বপূর্ণ, কারণ ধীরতম রিকোয়েস্টগুলোই গ্রাহককে অপেক্ষা করায়।
ডেটায় কোনো leakage ছিল? পরীক্ষা করেছ?	সময়ক্রম আর লেবেল আসার দিন মেনে leakage আটকেছি, unit test-ও আছে। একটা জানা leakage IEEE-CIS-এর নিজের লেবেলিংয়ে: fraud একই client-এর পরের লেনদেনেও ছড়ানো। Client ID বানিয়ে পরীক্ষা করেছি: এদের বাদ দিলে retraining-এর লাভ অর্ধেক হয় (০.০৫৬ থেকে ০.০২৭), কিন্তু নির্ভরযোগ্য থাকে, আর পদ্ধতিগুলোর ক্রম একই থাকে।